

## [4차 과제: V3.0] 딕셔너리와 파일 입출력을 활용한 데이터 고도화

### 🔥 4차 과제 업데이트 내용:

이번 업데이트의 핵심은 구조화된 데이터 관리를 위해 중첩 리스트 구조를 **딕셔너리(Dictionary)** 자료형으로 전환하고, 프로그램이 종료되더라도 데이터가 유실되지 않도록 **파일 입출력(File I/O)** 기능을 도입하여 시스템의 영속성을 확보하는 것입니다.

- **딕셔너리 기반 데이터 구조화:** 대원의 정보를 인덱스(0, 1, 2...)가 아닌 Key-Value 구조( `name` , `oxygen` , `rank` 등)로 명확하게 매핑하여 코드의 가독성을 높였습니다.
- **파일 입출력을 통한 데이터 보존:** 프로그램 시작 시 외부 텍스트 파일( `crew_data.txt` )에서 기존 대원 명부를 자동으로 불러오고, 새로운 대원이 등록되거나 프로그램이 종료될 때 변경사항을 파일에 실시간 저장하도록 구현했습니다.
- **고도화된 리스트 조작:** 전체 탐사대 데이터셋에서 특정 등급(예: S등급) 대원들만 필터링하거나, 정렬 알고리즘 및 내장 함수( `len` , `max` , `sum` , `sort` 등)를 4종 이상 복합적으로 활용하여 통계 시스템을 한 단계 더 발전시켰습니다.

### 🤖 AI 파트너십 과정

#### 내용 1: 리스트에서 딕셔너리로의 자료구조 전환 자문

- **프롬프트 요약:** "기존에 `member[0]` , `member[4]` 처럼 숫자로 데이터를 가져오다 보니 코드가 복잡해지면 헷갈려. 대원 이름이나 생존 지수를 글자로 직관적으로 관리할 수 있는 자료구조와 활용법을 알려줘."
- **적용 내용:** AI가 제안한 '딕셔너리(Dictionary)와 Key 활용법'을 채택함. 각 대원의 정보를 `{ "name": name, "survival_index": score }` 형태의 딕셔너리로 묶고, 이를 전체 탐사대 리스트에 담은 '리스트 안의 딕셔너리' 구조를 설계하여 데이터 접근성과 관리 효율을 극대화함.

#### 내용 2: 외부 파일 입출력(File I/O) 및 데이터 파싱 로직 설계

- **프롬프트 요약:** "프로그램을 켜다 켜도 이전에 등록한 우주 대원들의 데이터가 그대로 유지되게 만들고 싶어. 오픈소스나 텍스트 파일을 활용해서 데이터를 저장하고 다시 읽어오는 로직을 추천해줘."

### 🔧 Troubleshooting & 기술 회고

#### 문제 1: 파일 읽기 시 데이터 유실 및 줄바꿈 기호 포함 오류 (IndexError / 문자열 깨짐)

- **원인:** 텍스트 파일에서 대원 데이터를 한 줄씩 읽어올 때, 줄 끝에 붙는 개행 문자( `\n` )가 제거되지 않은 채로 형변환( `int` , `float` )이 시도되어 에러가 나거나 문자열 데이터 뒤에 공백이 붙는 현상 발생.
- **해결:** 파일을 읽어와 데이터를 분리하기 전, 문자열 양 끝의 공백과 개행 문자를 깔끔하게 지워주는 `.strip()` 메서드를 먼저 적용한 후 `.split(',')` 을 실행하여 안정적으로 데이터를 파싱함.

#### 문제 2: 파일이 존재하지 않을 때 발생하는 시스템 다운 현상 (FileNotFoundError)

- **원인:** 프로그램을 처음 실행하거나 데이터 저장 파일( `crew_data.txt` )이 아직 생성되지 않은 상태에서 무작정 로드 함수를 호출하여 파일이 없다는 치명적 오류와 함께 시스템이 비정상 종료됨.
- **해결:** 파일 읽기를 시도하기 전, 파일 존재 여부를 체크하거나 파이썬의 예외 처리 문법인 `try-except FileNotFoundError` 구조를 설계하여 파일이 없을 경우 에러로 멈추는 대신 "새로운 명부 파일을 생성합니다"라는 안내와 함께 빈 저장소를 안정적으로 개설하도록 조치함.